

Rapport de Projet



Gaël Kerninon
Thibault Bichet
Baptiste Vrbanc
Ethan Rouimi

Projet OCR

15 décembre 2025

Contents

I	Informations Générales	3
1	Informations Générales	4
1.1	Informations sur le Projet	4
1.1.1	Résumé	4
1.1.2	Membres	4
1.1.3	Répartition des Tâches	5
1.1.4	Chronologie des Tâches	5
1.2	Informations sur l'Implémentation	7
1.2.1	Outils et Logiciels	7
1.2.2	Architecture	7
II	Rapports Hebdomadaires	8
2	Rapports Hebdomadaires	9
2.1	Semaine du 22 septembre 2025	9
2.2	Semaine du 29 septembre 2025	10
2.3	Semaine du 6 octobre 2025	13
2.4	Semaine du 13 octobre 2025	17
2.5	Semaine du 20 octobre 2025	19
2.6	Semaine du 27 octobre 2025	22
2.7	Semaine du 3 novembre 2025	25
2.8	Semaine du 10 novembre 2025	27
2.9	Semaine du 17 novembre 2025	28
2.10	Semaine du 24 novembre 2025	31
2.11	Semaine du 1 décembre 2025	33
2.12	Semaine du 8 décembre 2025	35
III	Bibliographie et Sources	37
3	Bibliographie et Sources	38
3.1	Structures de Données	38
3.2	Algorithmes	38
3.3	Tutoriels et Ressources Vidéo	39
3.4	Librairies	39

Part I

Informations Générales

Chapter 1

Informations Générales

Introduction

Le projet **OCR KeltIA** s'inscrit dans le cadre du module de projet du semestre 3 à l'*EPITA*. Réalisé en équipe de quatre étudiants, il a pour objectif la conception et l'implémentation d'un logiciel de reconnaissance optique de caractères (OCR) capable d'analyser et de résoudre automatiquement une grille de mots cachés à partir d'une image. Ce projet associe plusieurs domaines fondamentaux de l'informatique : le traitement d'image, l'intelligence artificielle, la reconnaissance de formes et le développement bas-niveau en langage C.

Notre démarche s'articule autour de la rigueur, de la modularité et de la performance, en respectant les contraintes imposées par le cahier des charges du projet **OCR Word Search Solver**.

1.1 Informations sur le Projet

1.1.1 Résumé

Le projet vise à développer un système complet de reconnaissance de texte à partir d'images, intégrant des modules de prétraitement, de segmentation, de reconnaissance de caractères et de résolution automatique d'une grille de mots cachés. L'application, écrite en langage C, repose sur une architecture modulaire composée de plusieurs sous-systèmes : détection des zones de texte, découpage des caractères, entraînement et inférence d'un réseau de neurones artificiels, puis résolution logique de la grille. En sortie, le programme génère une image annotée contenant la grille résolue. Ce projet met en avant des compétences en algorithmique, en programmation système et en apprentissage automatique, tout en respectant les bonnes pratiques de développement collaboratif via *Git*.

1.1.2 Membres

Le groupe est composé des quatre étudiants suivants :

- Gaël Kerninon
- Thibault Bichet
- Ethan Rouimi
- Baptiste Vrbanac

Chaque membre participe activement à la conception, à l'implémentation et à la documentation du projet, en se répartissant les tâches selon les domaines de compétence de chacun. Les détails de ces tâches sont décrits dans la seconde partie.

1.1.3 Répartition des Tâches

Cette section a pour objectif de détailler la répartition des responsabilités au sein de l'équipe et de mettre en évidence la nature des contributions qui ont structuré l'avancement du projet. Les détails sur ces tâches sont restitués dans la section du rapport dédiée aux rapports individuels page 9. Le travail d'équipe sur des tâches communes ou lors de la maintenance du code source du projet ont également constitué une part importante des activités de l'équipe, qui est un travail à souligner.^{1 2}

Tâche	Responsable
Algorithme d'Upscale	Gaël Kerninon
Détection des Caractères	Thibault Bichet
Détection des Zones	Gaël Kerninon
Documents de Projet	Gaël Kerninon
Gestion de Projet	Ethan Rouimi
Grayscale	Baptiste Vrbanc
Interface Utilisateur	Thibault Bichet
Outils CI/CD	Ethan Rouimi
Réseau de Neurones	Ethan Rouimi
Rotation Automatique	Baptiste Vrbanc
Rotation d'Image	Baptiste Vrbanc
Solver	Gaël Kerninon
Suppression du Bruit	Baptiste Vrbanc

Table 1.1: Liste des Tâches² Individuelles par Ordre Alphabétique

1.1.4 Chronologie des Tâches

Cette section a pour objectif de détailler l'avancement du projet. Pour visualiser l'évolution du travail au fil des semaines, nous présentons la chronologie des tâches. Cette frise met en perspective les périodes d'activité pour chaque fonctionnalité et montre comment les efforts de Gaël, Baptiste, Ethan et Thibault se sont articulés depuis septembre jusqu'à la phase finale du projet en décembre. Elle illustre à la fois les développements séquentiels et le travail parallèle effectué par les différents membres.

Tourner la page S.V.P.

¹Rotation d'Image **Manuelle** puis **Automatique**

²Le temps de développement nécessaire varie d'une tâche à l'autre.

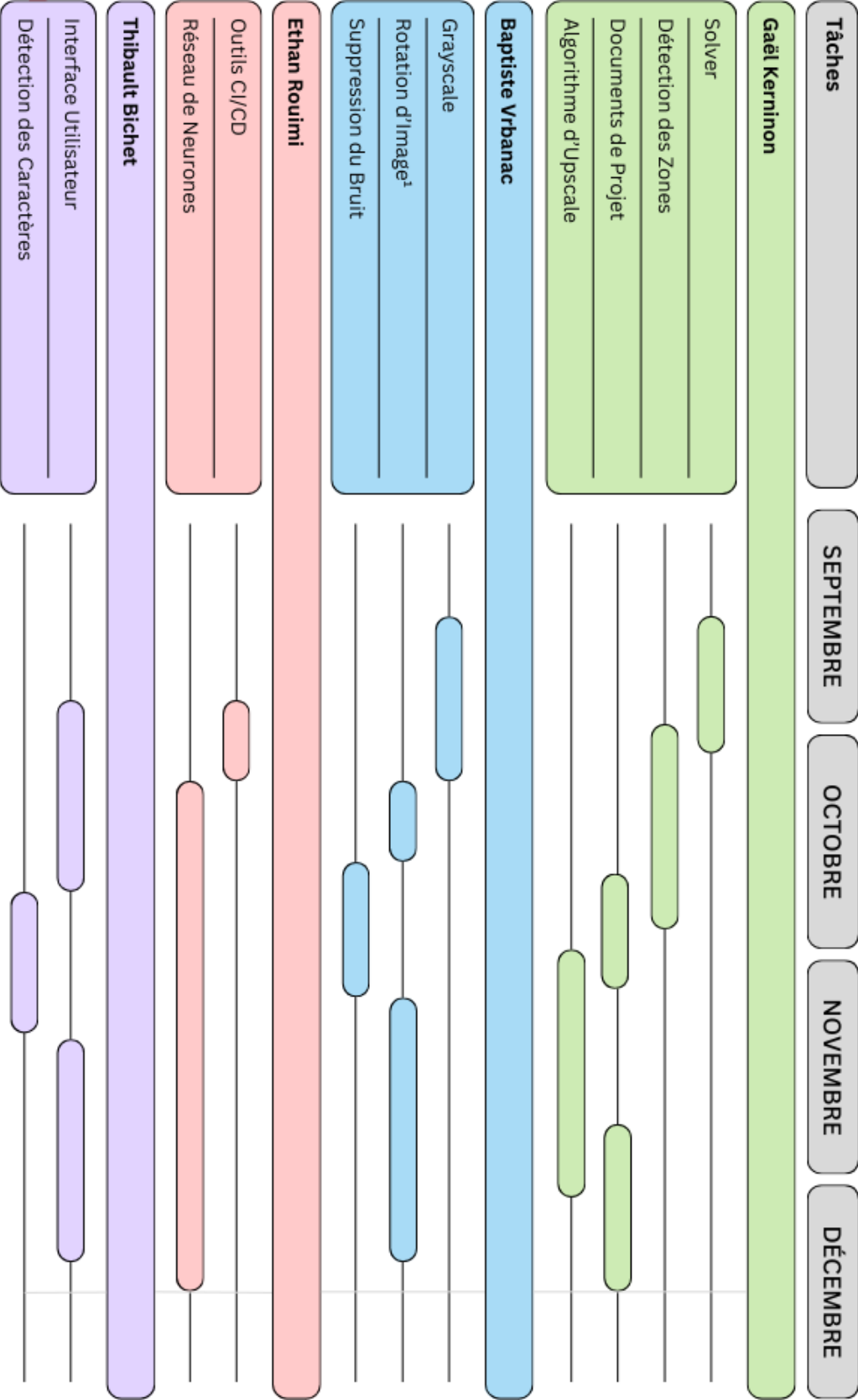


Figure 1.1: Chronologie des Tâches par Membre en fonction du Temps

1.2 Informations sur l'Implémentation

1.2.1 Outils et Logiciels

Le développement du projet s'effectue principalement sous environnement GNU/Linux, à l'aide d'outils et de logiciels libres :

1. Langage de programmation : C (standard C99)
2. Compilateur : GCC, avec des options comme `-Wall`, `-Wextra`, etc. pour garantir la robustesse du code
3. Environnements de développement : Vim et Visual Studio Code
4. Système de build : Makefile
5. Formatage automatique : `clang-format` via un hook Git pré-commit
6. Gestion de versions : Git (hébergement sur dépôt commun sur GitHub)
7. Rédaction des documents de projet : L^AT_EX

1.2.2 Architecture

L'architecture du projet est organisée selon une structure claire, permettant une séparation nette entre les différentes composantes logicielles. Le répertoire principal `KeltIA/` contient notamment :

- `src/` : les fichiers sources organisés par modules³ ;
- `include/` : les fichiers d'en-tête correspondants.
- `bins/` : les exécutables produits lors de la compilation ;
- `ressources/` : les jeux d'images et les données d'apprentissage nécessaires à la reconnaissance optique ;
- `scripts/` : les scripts d'automatisation et les *Git hooks* utilisés pour le formatage et la gestion du code ;
- `Makefile` : la description des règles de compilation et des dépendances du projet.

L'ensemble est conçu pour permettre une évolution progressive du projet, en facilitant la phase de développement pour les membres du groupe. Cette organisation modulaire garantit la maintenabilité du code, la clarté des dépendances et la cohérence globale du système. De plus, nous avons également ajouté un `README` rédigé en *Markdown* à la source du projet pour aider à se familiariser avec le projet.

³Traitement d'image, réseau de neurones, résolution de grille, etc.

Part II

Rapports Hebdomadaires

Chapter 2

Rapports Hebdomadaires

2.1 Semaine du 22 septembre 2025

Réunion de Projet (Baptiste, Gaël, Thibault, Ethan)

Le groupe a organisé deux réunions afin de planifier le déroulement du projet et répartir les différentes tâches entre les membres. La première réunion s'est tenue en présentiel, tandis que la seconde a été réalisée en visioconférence. Lors de ces réunions, le planning d'avancement a été défini, et les responsabilités de chacun ont été précisées : Gaël sera en charge des programmes Solver ainsi que de la détection des zones dans une image, Ethan s'occupera du réseau de neurones, tandis que Baptiste prendra en main les traitements d'image tels que la rotation, la conversion en niveaux de gris et la suppression de bruit. Enfin, Thibault développera l'interface utilisateur. Cette organisation a permis d'assurer une répartition claire des responsabilités et de suivre efficacement l'avancement du projet.

Solver (Gaël)

Cette semaine, Gaël a commencé la tâche du solver qui consiste à implémenter un algorithme permettant de trouver un mot dans une grille de lettres. Cet algorithme sera dans un premier temps implémenté en `Python`, puis sera traduit en langage `C`, afin d'assurer une bonne compréhension et réflexion autour de l'algorithme avant sa version définitive dans le projet. L'algorithme principal est encapsulé dans la fonction : `solver(grid, word)`. On itère la grille de lettres à la recherche du mot, pour chaque lettre, et pour chaque direction¹, les lettres de la grille sont comparées successivement à celles du mot, si toutes les lettres correspondent, la fonction retourne les coordonnées de début et de fin du mot, et renvoie `None` sinon. L'algorithme s'arrête automatiquement dès qu'un caractère ne correspond pas ou que les limites de la grille sont dépassées. Ce script inclut également des tests unitaires permettant de vérifier la recherche de mots horizontaux, verticaux et diagonaux, ainsi que la gestion des cas où le mot n'est pas présent, assurant ainsi la fiabilité et la flexibilité du solver pour des grilles de taille modérée. On notera que pour une grille de dimensions $n \times m$ et un mot de longueur l , la complexité de l'algorithme est de l'ordre de $O(n \times m \times l)$. La prochaine étape du projet consistera à implémenter cet algorithme en langage `C`.

À FAIRE :

- Solver (Gaël) : Implémenter le solver en langage `C` et le tester.
- Grayscale (Baptiste) : Implémenter le grayscale en langage `C`.
- Réseau de Neurones (Ethan) : Implémenter le `XNOR` avec un réseau de neurones.
- Interface Utilisateur (Thibault) : Trouver la meilleure bibliothèque pour créer des UI.

¹Solver : Les 8 directions dans lesquelles le solver cherche un mot sont N, S, E, W, NW, NE, SE, SW

2.2 Semaine du 29 septembre 2025

Grayscale (Baptiste)

La première tâche de Baptiste dans ce projet consistait à implémenter une conversion d'une image en couleur vers une image en niveaux de gris (**Grayscale**). Cette tâche vise à simplifier le traitement des images à l'avenir en supprimant les informations de couleur, afin de ne conserver que les intensités lumineuses des pixels. En résumé, l'objectif de cette première étape est de convertir une image en couleur en une image en noir et blanc.

Dans un premier temps, Baptiste a opté pour l'algorithme suivant : on calcule la moyenne des intensités des pixels de l'image, cette moyenne servant de seuil de binarisation. Chaque pixel dont l'intensité était inférieure ou égale au seuil était alors transformé en noir, et les autres en blanc. L'algorithme initial reposait donc sur le calcul de la moyenne globale de l'image : on calcule la somme des composantes RGB² de tous les pixels, que l'on divise par le nombre total de pixels. Cependant, cette méthode a révélé un problème d'efficacité. Il a donc fallu opter pour une autre méthode.

Par la suite, Baptiste a mis en place un second algorithme, fondé sur une formule intégrant les valeurs minimale (Min_i) et maximale (Max_i) d'intensité de l'image. La formule utilisée est la suivante :

$$Min_i + (Max_i - Min_i) \times 0.1$$

Cette méthode s'est montrée plus adaptée, mais elle demeure peu efficace pour les images présentant une luminosité trop faible ou, au contraire, trop élevée, due à la présence de pixels respectivement noirs ou blancs.

Outils CI/CD (Ethan)

Cette semaine, Ethan a travaillé sur la mise en place d'outils de CI/CD³ afin de faciliter le développement et la maintenance du projet. Un système de tests automatisé en langage C a été conçu pour vérifier le bon fonctionnement des différents modules à chaque mise à jour du code, garantissant ainsi la stabilité du projet au fil des développements.

²RGB : Abbréviation pour *Red*, *Green*, *Blue*, *composantes de la couleur d'un pixel en synthèse additive*.

³(Intégration Continue et Déploiement Continu)

Solver (Gaël)

La semaine suivante, Gaël a effectué l'implémentation du solveur en langage C, en conservant la logique de recherche utilisée précédemment en Python. La fonction principale `solver` parcourt chaque case de la grille et explore les huit directions possibles afin de détecter le mot à rechercher. Les coordonnées de la grille sont représentées sous forme d'un tableau linéarisé, ce qui permet de calculer les indices d'accès en utilisant `grid[i * width + j]`. Pour chaque direction, les lettres sont comparées successivement et, si toutes correspondent, les coordonnées de début et de fin du mot sont affichées, sinon, le programme indique que le mot n'a pas été trouvé. Cette implémentation conserve la même complexité algorithmique que la version Python, à savoir $O(n \times m \times l)$ pour une grille de dimensions $n \times m$ et un mot de longueur l , et bénéficie des performances accrues du langage C pour les grilles de taille plus importante ou les traitements en temps réel. Les tests ont été réalisés et vérifient bien les attendus. Cette tâche est désormais considérée comme aboutie et sera merge dans la branche `main` dans les jours suivants.

HORIZONTAL	
DXRAHCLBGA	((0, 0), (9, 0)) HORIZONTAL
DIKCILEOKC	((9, 7), (9, 0)) VERTICAL
IGAJHYLYHI	((0, 1), (7, 8)) DIAGONAL
HGFGODTIOT	((4, 8), (1, 5)) FIND
GDLROWKBFR	((9, 8), (5, 8)) HELLO
PLNRDNERGE	((5, 5), (1, 5)) WORLD
JHAIDUAJGV	((8, 1), (1, 8)) GOLDORAK
UKGFFOLLEH	None EPITA

Figure 2.1: Exemple d'Application du Solver sur une Grille

Interface Utilisateur (Thibault)

Le premier pas dans la création de l'interface graphique, a été pour Thibault de choisir quel architecture utiliser. La bibliothèque choisie est `GTK 3.0`. Cette bibliothèque était selon nous meilleur choix car elle est largement supportée, y compris par le PIE, elle permet de créer facilement des interfaces modernes et intuitives à partir d'une bibliothèque de composant. Également, cette bibliothèque permet de s'affranchir des difficulté syntaxiques du C en permettant développeurs à utiliser un fichier `XML` pour créer l'arbre de composants et leur attribuer des propriétés d'affichage. Nous avons choisi d'utiliser le logiciel `Glade` pour générer le fichier `XML` contenant les interfaces.

Réseau de Neurones (Ethan)

En parallèle de l'ajout des Outils CI/CD, Ethan a commencé la conception des premières structures dédiées aux réseaux de neurones, en définissant notamment les structures (**Layer**) et le **Neuronal Network**. Ces structures permettent de représenter les couches d'un réseau et de réaliser les calculs nécessaires à sa propagation. Ethan a également implémenté un mécanisme de sérialisation sous la forme d'un fichier personnalisé au format **.nn**, permettant d'enregistrer et de recharger des modèles de réseaux de neurones. Cette approche assure la persistance des modèles et facilite leur réutilisation ou leur transfert entre différents environnements d'exécution.

Enfin, un premier prototype fonctionnel (POC) de réseau de neurones a été développé pour résoudre le problème logique du **NXOR**⁴, dont la porte est présente ci-contre. en s'appuyant sur des perceptrons. Ce test a permis de valider les principes de base du calcul neuronal et de confirmer le bon fonctionnement des structures et mécanismes de sauvegarde précédemment mis en place.

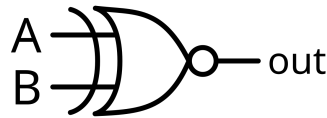


Figure 2.2: Une Porte XNOR

```

=====
                        Save and Load NNs
=====
[PASS] write nn
[PASS] load nn
[PASS] write nn, load

=====
                        XNOR NN for demo
=====

Testing XNOR function:
✓ 0 XNOR 0 = 1.00 (expected 1)
✓ 0 XNOR 1 = 0.00 (expected 0)
✓ 1 XNOR 0 = 0.00 (expected 0)
✓ 1 XNOR 1 = 1.00 (expected 1)
[PASS] test XNOR nn

```

Figure 2.3: Résultats du Réseau de Neurones (*perceptrons*) pour la porte XNOR

À FAIRE :

- Grayscale (Baptiste) : Envisager d'utiliser l'algorithme d'Otsu.
- Détection des Zones (Gaël) : Réfléchir à l'algorithme pour détecter les zones d'une image.
- Réseau de Neurones (Ethan) : Continuer les recherches sur le réseau de neurones.
- Interface Utilisateur (Thibault) : Créer la structure de l'interface sur Glade.

⁴NXOR ou XNOR : $\overline{A \oplus B} = \overline{A} \cdot \overline{B} + A \cdot B$

2.3 Semaine du 6 octobre 2025

Détection des Zones (Gaël)

Pour cette étape, l'objectif était de réduire la résolution d'une image binaire, c'est-à-dire ne contenant que des 0 et des 1 (0 représentant les pixels blancs et 1 les pixels noirs) afin de faciliter la détection des zones contenant des informations significatives, comme la grille ou la liste de mots. Le programme mentionné ci-dessous, a été implémenté, de même que l'était le solver, en langage Python, et sera traduit en langage C ultérieurement. L'algorithme utilisé repose sur un partitionnement de la matrice **matrix** de l'image en une grille **grid** de dimensions $n \times n$. Chaque case de cette grille correspond à un bloc de pixels de l'image d'origine, et sa valeur est déterminée en fonction de la proportion de pixels noirs qu'il contient en suivant la formule suivante :

$$\text{grid}[i][j] = \begin{cases} 1, & \text{si } \frac{\text{Nombre de Pixels Noirs}}{\text{Nombre Total de Pixels}} \geq \text{seuil} \\ 0, & \text{sinon} \end{cases}$$

On notera que :

- $\text{seuil} = 0.0 \implies$ Au moins un pixel noir suffit pour considérer le bloc comme noir
- $\text{seuil} = 0.5 \implies$ Une majorité de pixels noirs suffit pour considérer le bloc comme noir
- $\text{seuil} = 1.0 \implies$ Ne considérera jamais le bloc comme noir

Le découpage s'adapte automatiquement si la dimension de l'image n'est pas un multiple exact de n , en répartissant les pixels « restants » sur les premières lignes et colonnes de la grille. L'algorithme renvoie finalement une nouvelle matrice $n \times n$ représentant l'image réduite, où chaque cellule indique la présence ou l'absence d'une zone noire significative. Cette méthode, appelée **downscale**, permet de simplifier le traitement ultérieur pour la détection de zones, tout en conservant les informations structurantes de l'image.

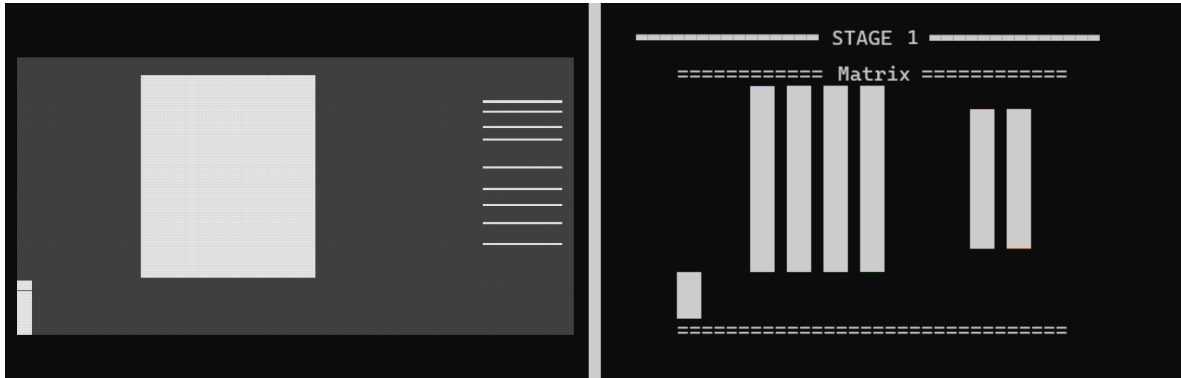


Figure 2.4: Exemple de *Downscale* d'une Image de 600×400 en une grille de 10×10 avec $\text{seuil} = 1\%$

Grayscale (Baptiste)

Pour résoudre le problème de la précédente méthode, inefficace pour les images présentant une luminosité trop faible ou trop élevée, Baptiste a finalement opté pour l'utilisation de l'algorithme d'Otsu. Cet algorithme s'appuie sur l'histogramme des intensités de l'image, c'est-à-dire sur la répartition du nombre de pixels pour chaque niveau de gris. Il permet de déterminer automatiquement le seuil optimal de binarisation en minimisant la variance entre les pixels clairs et foncés, offrant ainsi une bien meilleure adaptabilité aux variations de luminosité de l'image.

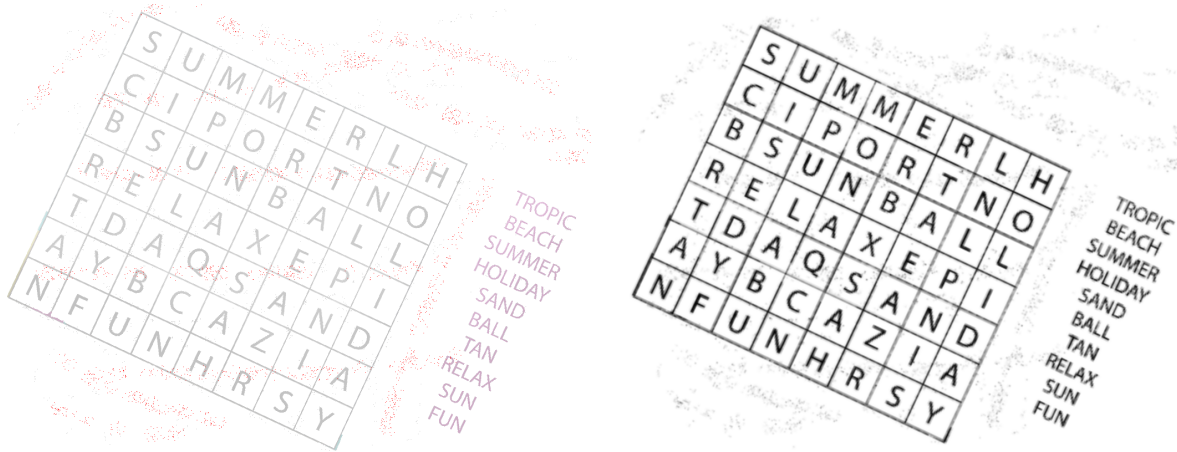


Figure 2.5: Exemple d'Application de l'Algorithme de Grayscale (Avant / Après)

De plus, Baptiste a utilisé la librairie **MagickWand** afin d'effectuer certaines retouches sur l'image. Cet outil permet de sélectionner et de modifier des zones précises de l'image en fonction de la similarité des pixels, ce qui facilite le nettoyage et l'amélioration du rendu final.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a poursuivi ses recherches sur le fonctionnement des réseaux de neurones et à la compréhension des différents types de neurones utilisés dans les architectures classiques. Il a ensuite implémenté des neurones à fonction d'activation sigmoïde, permettant d'introduire une non-linéarité dans le modèle et de rendre possible l'apprentissage de relations complexes entre les entrées et les sorties. Cette étape a constitué une base essentielle pour la mise en place d'un futur réseau de neurones plus complet.

Solver (Ethan, Gaël)

Cette semaine, des efforts significatifs ont été déployés pour consolider et améliorer l'interface d'entrée du module de résolution (*solver*). Ethan a notamment pris en charge l'implémentation d'une nouvelle fonction, permettant de lire, de manière dynamique, une matrice de données directement à partir d'un fichier texte externe. Cette fonctionnalité est importante pour les tests, car elle standardise les procédures et offre une flexibilité accrue pour l'injection des matrices à résoudre. Par la suite, Gaël et Ethan ont travaillé ensemble pour intégrer cette nouvelle fonction de chargement au cœur de la procédure principale `main()` du `solver`. Cette intégration a été réalisée de manière à ce que l'exécutable puisse désormais accepter et traiter des chemins de fichiers spécifiés en arguments de la ligne de commande, assurant ainsi une exécution robuste et entièrement paramétrable en fonction des scénarios de validation.

Interface Utilisateur (Thibault)

Cette semaine, Thibault s'est consacré à la création de l'UI⁵ sur glade. Avant de créer cette UI, il a fallu définir quels étaient les contrôles essentiels à ajouter à l'interface utilisateur. Ces éléments sont :

- Un bouton pour charger une image
- Un bouton pour charger un modèle
- Un bouton pour lancer la résolution de grille

Il a également fallu choisir comment les disposer. Pour cela nous avons opté pour une barre latérale à gauche contenant tous les paramètres ainsi que les boutons précédemment cités et à droite l'image en cours d'utilisation. L'image par défaut est le logo du groupe, cette solution permet grâce aux paramètres d'extension de la fenêtre, de conserver une taille de fenêtre appropriée et de montrer à l'utilisateur où va se situer le "viewport".

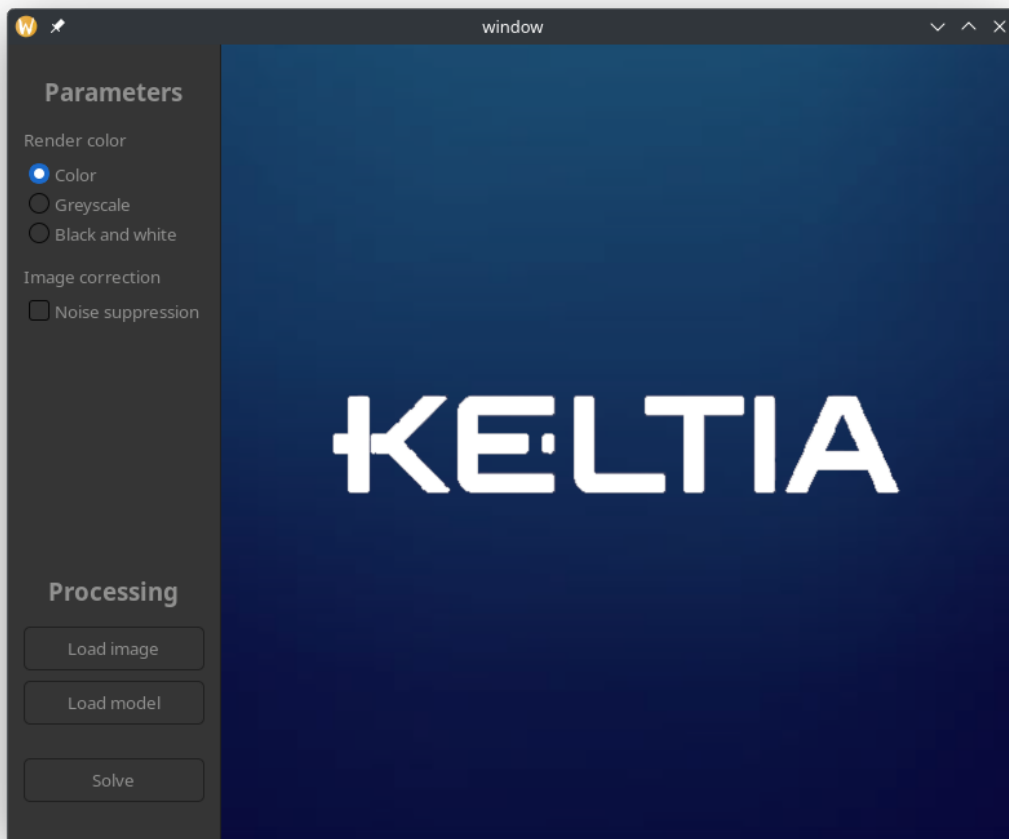


Figure 2.6: Aperçu de l'Interface Utilisateur

⁵UI (*User Interface*) : Interface Utilisateur

À FAIRE :

- Rotation d'Image (Baptiste) : Faire des recherches sur l'implémentation de la rotation.
- Détection des Zones (Gaël) : Continuer l'algorithme pour qu'il détecte deux zones dans une grille réduite.
- Réseau de Neurones (Ethan) : Faire des recherches sur les méthodes d'apprentissages.
- Interface Utilisateur (Thibault) : Code behind et débogage **GTK**.

2.4 Semaine du 13 octobre 2025

Rotation d'Image (Baptiste)

Cette semaine, Baptiste s'est consacré à l'implémentation de la rotation manuelle de l'image en fonction d'un angle donné. Il a de nouveau utilisé la librairie **MagickWand** pour réaliser des transformations supplémentaires sur les images concernant les rotations manuelles. Cette fonctionnalité permet de spécifier précisément l'angle de rotation souhaité (en degrés) et d'appliquer la transformation directement à l'image. L'utilisation de **MagickWand** s'est donc révélée pratique pour effectuer ces opérations de manipulation d'image grâce aux fonctions intégrées de la bibliothèque, sans nécessiter d'implémentation complexe supplémentaire.

```
#include <MagickWand/MagickWand.h>
#include <stdio.h>
#include <stdlib.h>

MagickWand *rotate_image(const MagickWand *input_wand, double angle)
{
    MagickWand *rotated_wand = NULL;
    MagickBooleanType status;
    PixelWand *background = NULL;

    // Clone the input image
    rotated_wand = CloneMagickWand(input_wand);
    if (rotated_wand == NULL)
    {
        fprintf(stderr, "Error: unable to clone the input MagickWand.\n");
        return NULL;
    }

    // Define the background color (white)
    background = NewPixelWand();
    PixelSetColor(background, "white");

    // perform the rotation (only 3 arguments)
    status = MagickRotateImage(rotated_wand, background, angle);
    if (status == MagickFalse)
    {
        fprintf(stderr, "Error: failed to rotate the image.\n");
        if (rotated_wand) rotated_wand = DestroyMagickWand(rotated_wand);
        if (background) background = DestroyPixelWand(background);
        return NULL;
    }

    // cleanup
    if (background) background = DestroyPixelWand(background);

    return rotated_wand;
}
```

Figure 2.7: Code de la fonction `rotate_image()`

Réseau de Neurones (Ethan)

Cette semaine, Ethan s'est d'abord concentré sur la recherche théorique sur les réseaux de neurones et sur les méthodes d'apprentissage associées. L'objectif principal était de consolider la compréhension des fondements nécessaires à l'implémentation d'un réseau de neurones dans le cadre du projet. Ces connaissances serviront de base pour le développement futur du module d'intelligence artificielle du projet.

Détection des Zones (Gaël)

Cette semaine, la tâche de Gaël consiste à identifier les deux zones principales dans une image binaire, après application du partitionnement (**downscale**). L'algorithme prend en entrée une matrice où chaque case vaut 0 (blanc) ou 1 (noir), et retourne les coordonnées des deux plus grands rectangles de pixels noirs. Le principe repose sur une parcours en profondeur (DFS) de la matrice :

1. Chaque cellule noire non visitée initialise une pile qui permet d'explorer toutes les cellules connectées à celle-ci (voisinage 4-connecté : haut, bas, gauche, droite).⁶
2. Pendant l'exploration, l'algorithme suit les coordonnées minimales et maximales atteintes afin de délimiter un rectangle englobant tous les pixels noirs connectés.
3. Une fois tous les pixels d'une zone visités, le rectangle est ajouté à une liste.
4. Après avoir identifié tous les rectangles présents dans la matrice, l'algorithme calcule l'aire de chacun et retient uniquement les deux plus grands. Les coordonnées retournées correspondent aux coins supérieur gauche et inférieur droit de chaque rectangle.

Cette méthode permet de détecter efficacement les deux zones importantes dans une image, tout en éliminant les petits regroupements de pixels isolés qui ne correspondent pas aux zones d'intérêt principales.

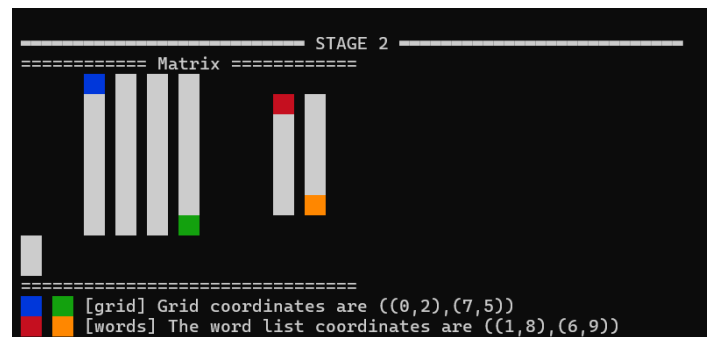


Figure 2.8: Exemple d'Identification des deux Zones

Gestion de Projet (Ethan)

En parallèle, un travail de restructuration du dépôt Git a été entrepris. L'objectif était de rendre le projet plus modulaire, plus facile à compiler et mieux organisé pour accueillir de nouveaux modules (notamment le réseau de neurones et les outils de traitement d'image). Cette tâche a impliqué une révision de l'arborescence des répertoires, une uniformisation des chemins d'inclusion, ainsi qu'une mise à jour des scripts de compilation (**Makefile**). Plusieurs difficultés ont été rencontrées lors de cette étape, notamment des conflits Git complexes dus à des divergences entre branches. Leur résolution a nécessité une intervention minutieuse pour préserver l'intégrité du code et assurer la compatibilité entre les différents modules.

À FAIRE :

- Suppression du Bruit (Baptiste) : Écrire le script permettant de supprimer le bruit sur une image.
- Détection des Zones (Gaël) : Terminer l'algorithme et l'implémenter en langage C.

⁶Cette méthode permet d'assurer la détection d'une zone même si elle contient des trous (coins manquants, espacement entre les lettres, ...). Cela rajoute de la fiabilité à l'algorithme dans des cas non-triviaux.

2.5 Semaine du 20 octobre 2025

Suppression du Bruit (Baptiste)

Ce script a pour objectif de supprimer le « bruit » présent sur une image à l'aide de fonctions intégrées à la librairie MagickWand. Le principe repose sur l'application d'un filtre médian sur l'image, qui consiste à remplacer la valeur de chaque pixel par la médiane des intensités des pixels situés dans un voisinage de taille $radius \times radius$. Cette méthode permet de réduire efficacement les imperfections tout en préservant les contours et les détails importants de l'image. Le rayon du voisinage (*radius*) est déterminé automatiquement par le programme, ce qui assure une adaptation du filtrage en fonction des caractéristiques de l'image traitée. L'aboutissement de ce script permettra de terminer celui de la détection des zones dont l'implémentation par Gaël est pratiquement achevée.

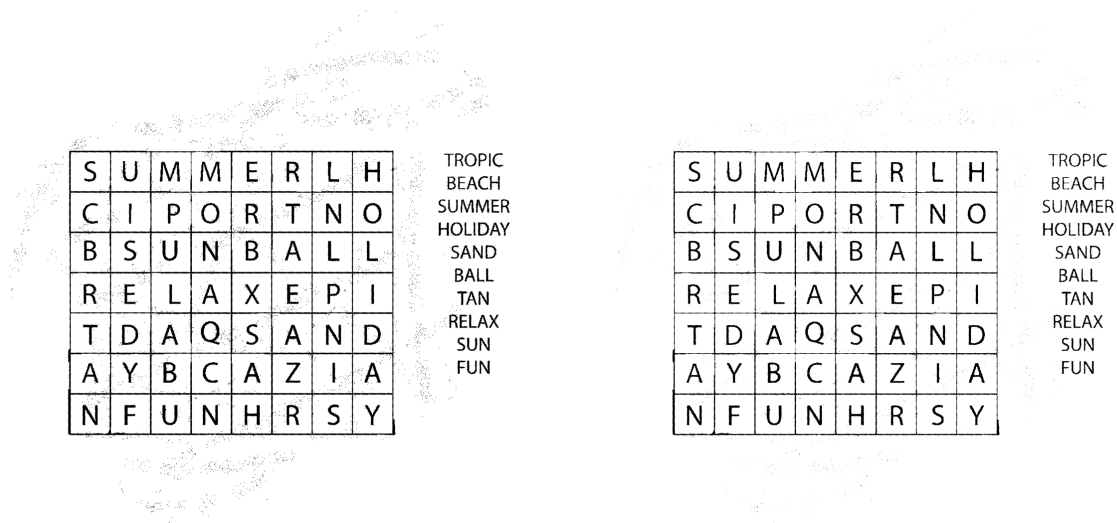


Figure 2.9: Exemple d'Application de l'Algorithme de Suppression du Bruit (Avant / Après)

Interface Utilisateur (Thibault)

Cette semaine a été consacrée au Code behind et débogage de GTK. Après la phase de création, il a fallu passer à la partie code. Le fichier `src/window/window.c` contiendra donc les fonctions essentielles à l'affichage et à l'interaction avec l'application. Thubault a créé dans ce fichier, un événement permettant de faire en sorte que la fermeture de la fenêtre principale entraîne aussi la fermeture de l'application. Nous avons fait le choix de ne pas connecter tout de suite les composants aux fonctions de notre application car celle-ci sont encore en modification en risque de changer. Ainsi cela évite de tout casser lors de la modification d'une fonction. Il a également fallu créer une `g_ressource` pour le logo de l'entreprise afin que celui-ci soit inclus dans le binaire du programme ce qui permet de le rendre portable et non dépendant d'un chemin de fichier absolu. Cependant nous avons remarqué un bug provenant de GTK, en effet, utiliser l'address sanitizer avec GTK est inutile car par défaut, la bibliothèque alloue de la mémoire sans la libérer avant la fin de l'exécution causant grand nombre de messages de memory leak n'étant pas provoqués par notre code. Nous avons donc désactivé `address sanitizer` pour les tests de la fenêtre.

Détection des Zones (Gaël)

Cette semaine, Gaël a terminé l'algorithme de détection des zones. La fonction principale prend en entrée une matrice de 0 et de 1, le nombre de subdivisions souhaité n pour la grille, ainsi qu'un **seuil** permettant de déterminer si une case de la grille doit être considérée comme noire. Le programme effectue successivement les étapes suivantes :

1. Partitionnement (**downscale**) de la matrice initiale en une grille $n \times n$. Chaque case représentant un bloc de pixels et valant 1 si la proportion de pixels noirs dépasse le seuil.
2. Détection des rectangles dans la grille réduite, pour identifier les deux zones principales sélectionnées en mesurant leurs aires respectives.
3. Conversion des coordonnées pour les reporter sur la matrice originale⁷, afin d'obtenir les limites exactes des zones détectées.

Cet algorithme a été entièrement réécrit en langage C, en conservant la même logique et les mêmes étapes que la version Python, tout en bénéficiant des performances accrues du C pour le traitement d'images de grande taille et pour les applications temps réel. Il permet ainsi d'obtenir automatiquement les coordonnées des zones d'intérêt et de préparer les données pour les étapes suivantes du programme. Un programme de tests a également été implémenté en langage C afin de servir de démonstration visuelle lors de la soutenance, affichant graphiquement dans le terminal des petites grilles modélisées à l'aide d'emojis de couleur, pour offrir une expérience visuelle illustrant l'algorithme décrit dans ce présent rapport.

Réseau de Neurones (Ethan)

Au cours de cette semaine, Ethan a approfondi ses connaissances sur le fonctionnement de la rétropropagation dans les réseaux de neurones artificiels. Ce mécanisme repose sur le calcul d'une fonction de coût, qui mesure l'écart entre les sorties prédites par le modèle et les valeurs attendues. L'objectif de la rétropropagation est de minimiser cette fonction de coût en ajustant progressivement les poids du réseau. Pour cela, l'algorithme applique la dérivée de la fonction de coût par rapport à chaque poids, grâce à la règle de la chaîne, permettant une propagation de l'erreur du neurone de sortie vers les couches précédentes. Ce processus est essentiel pour l'apprentissage supervisé, car il guide le modèle vers une amélioration de ses performances à chaque itération. Les formules de modification des poids et des biais utilisées sont les suivantes :

$$w_k \rightarrow w'_k = w_k - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial w_k}$$

$$b_l \rightarrow b'_l = b_l - \frac{\eta}{m} \sum_j \frac{\partial C_{X_j}}{\partial b_l}$$

La rétropropagation est donc l'algorithme essentiel de l'apprentissage supervisé des réseaux de neurones, puisqu'elle permet la minimisation de la fonction de coût par un ajustement méthodique des poids et des biais. Ce processus crucial s'opère par la propagation de l'erreur des neurones de sortie vers l'entrée grâce à l'application de la règle de la chaîne, guidant ainsi le modèle vers une amélioration continue de sa précision à chaque itération.

⁷On admet que puisque la **grid** est une version minimisée de la matrice **matrix**, les coordonnées de la grille dans la matrice sont proportionnelles à la largeur et hauteur d'un bloc de la matrice correspondant à une case de **grid**

À FAIRE :

- Documents de Projet (Gaël) : Rédaction des documents⁸ supports du projet pour l'équipe.
- Débogage (Baptiste, Gaël) : Vérification de la compatibilité des algorithmes de `grayscale()` et `remove_noise()` avec les algorithmes de `detect_zones()`
- Détection des Caractères (Thibault) : Faire l'algorithme.

⁸Documents : Rapport (*Ce présent document*), Plan de Soutenance

2.6 Semaine du 27 octobre 2025

Débogage (Baptiste)

Cette semaine, Baptiste a consacré du temps au débogage des scripts développés précédemment. L'objectif principal était de corriger les erreurs rencontrées lors de l'exécution des programmes, d'améliorer la robustesse des fonctions et de garantir un comportement fiable sur différents types d'images pour la soutenance. Cette phase de débogage a permis d'assurer que les algorithmes de conversion en niveaux de gris, de suppression du bruit et de rotation d'images fonctionnent correctement et de manière cohérente.

Débogage (Gaël)

Gaël a effectué cette semaine une relecture complète des algorithmes du solveur et de la détection des zones. Pour le solveur, il s'agissait principalement de vérifier la cohérence des indices dans les grilles, de s'assurer que les directions étaient correctement explorées et que les coordonnées retournées correspondaient bien aux conventions attendues. Pour la détection des zones, le débogage a porté sur le bon fonctionnement du `downscale`, la répartition correcte des pixels restants dans la grille, et essentiellement sur l'identification des deux rectangles principaux via l'algorithme `find_rectangles()`. En effet, des doutes subsistaient sur la fiabilité de cet algorithme dans des conditions réelles, qui, lors du développement, n'avait été testé que sur des cas parfaits, ou triviaux, attendant l'aboutissement du travail de Baptiste sur la suppression du bruit pour le tester dans les bonnes conditions. Cela a permis de s'assurer que les coordonnées renvoyées correspondaient aux zones réelles de l'image, et que les conversions entre la grille réduite et la matrice originale étaient exactes.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a réalisé l'implémentation complète de l'algorithme de rétropropagation (`back propagation`) et la mise en place du processus d'entraînement des réseaux de neurones. L'objectif principal était de concevoir un système générique et évolutif, capable de s'adapter à n'importe quelle architecture de réseau et à tout type de jeu de données, sans dépendre d'un modèle fixe. L'implémentation repose sur une structure modulaire permettant de définir dynamiquement le nombre de couches, le nombre de neurones par couche, ainsi que les fonctions d'activation associées. Afin de valider le bon fonctionnement du mécanisme d'apprentissage, un premier exemple concret a été réalisé : l'entraînement d'un petit réseau de neurones pour simuler le comportement de la porte logique NXOR, problème classique démontrant la capacité d'un réseau multicouche à modéliser des relations non linéaires.

Gestion de Projet (Ethan)

En parallèle du réseau de neurones, une refactorisation complète de la base de code liée aux réseaux de neurones a été effectuée. Cette restructuration a notamment permis de modulariser les différentes méthodes d'activation afin de faciliter leur utilisation et leur extension, d'intégrer directement la fonction d'activation à l'intérieur de chaque couche, assurant une meilleure encapsulation et une plus grande cohérence dans le flux de calcul et de rendre le système plus flexible, permettant désormais de changer la fonction d'activation d'une couche sans modifier la structure globale du réseau. Cette itération du développement consolide donc la partie "apprentissage" du projet, tout en préparant le terrain pour des tests sur des réseaux plus complexes et des ensembles de données réels.

Détection des Caractères (Thibault)

Pour la détection de caractère nous avons utilisé un algorithme de remplissage par diffusion (*flood fill*). Le principe est simple, on itère dans la grille pour trouver un pixel noir. Puis on se déplace grâce à une pile de pixel noir en pixel noir jusqu'à avoir détecté les pixel noirs adjacents. Ainsi on peut définir une zone du pixel supérieur gauche au pixel inférieur droit comme contenant un caractère. On enregistre ces coordonnées dans un tableau. Pour limiter les erreurs, on admet qu'un caractère ne pas pas faire moins de 4 pixels, ainsi on réduit le risque de détecter le bruit résiduel. Parfois la bordure de la grille est aussi détectée. Pour la supprimer, on compare sa taille aux autre éléments du tableau et on admet qu'un caractère ne peut pas être 2 fois plus grand que la moyenne, si c'est le cas on le supprime.

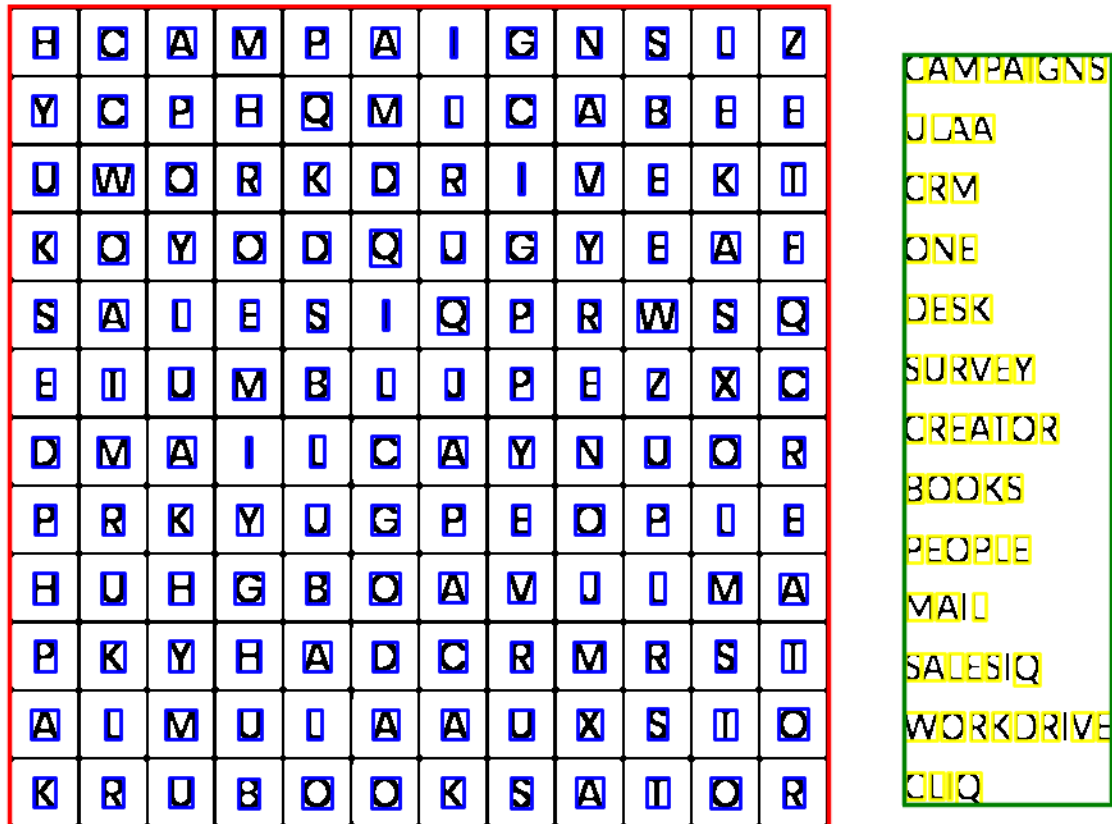


Figure 2.10: Exemple de la Détection des Zones et des Caractères

Documents de Projet (Gaël)

Cette semaine, Gaël a rédigé le rapport de projet en $\text{\LaTeX}$. Il s'est appliqué à structurer le document de manière claire et cohérente, en respectant les conventions et le format exigé. L'utilisation de $\text{\LaTeX}$ a permis d'obtenir une mise en page professionnelle, facilitant ainsi la lecture et la compréhension du contenu technique du rapport.

À FAIRE :

- Algorithme d'Upscale (Gaël) : Implémenter l'algorithme pour normaliser les dimensions.
- Rotation Automatique (Baptiste) : Réfléchir à l'implémentation de l'algorithme de rotation automatique.
- Réseau de Neurones (Ethan) : Implémenter la structure `ConvLayer`.
- Interface Utilisateur (Thibault) : Mettre en place les `Callback Functions`.

2.7 Semaine du 3 novembre 2025

Algorithme d’Upscale (Gaël)

Cette semaine, Gaël a commencé le développement de l’algorithme d’agrandissement (*upscale*). Cet algorithme est destiné à normaliser la taille des caractères extraits avant leur passage dans le réseau de neurones. L’objectif est donc de concevoir un qui prend en paramètres une matrice de pixels représentés par des 0 et des 1 représentant respectivement les pixels noirs et blancs, et qui renvoie une nouvelle matrice carrée de taille constante contenant le même modèle que la matrice d’entrée. Cet algorithme sera d’abord implémenté en `Python`, afin de tester la logique et d’obtenir un prototype facile à tester avant l’implémentation finale en langage `C`.

L’algorithme repose sur un redimensionnement de type *nearest-neighbor*, où chaque cellule de la matrice finale récupère la valeur du pixel correspondant dans la matrice d’origine suivant un ratio d’échelle horizontal et vertical. Plusieurs tests ont été réalisés sur des matrices carrées simples (3x3, 5x5, motifs carrés ou diagonaux), permettant d’ajuster la méthode de calcul des indices et de vérifier que la proportion des caractères était conservée. Avec cette première version en `Python`, l’algorithme a permis de confirmer la faisabilité de la fonction et de définir clairement les structures de données nécessaires pour la future traduction en langage `C`.

Rotation Automatique (Baptiste)

Cette semaine, Baptiste a commencé la tâche de rotation automatique. Cette tâche a pour objectif de détecter et de corriger automatiquement l’inclinaison d’une image avant son passage dans les étapes suivantes pour la reconnaissance. Cette phase est essentielle pour garantir la qualité du pré-traitement, notamment dans le cadre où une image inclinée dégrade fortement les performances des algorithmes de détection et donc du réseau de neurones. L’implémentation repose sur la bibliothèque `MagickWand`, utilisée pour charger, manipuler et transformer les images. Le système combine deux stratégies de détection complémentaires : l’analyse des contours et la projection horizontale.

Cette semaine a donc principalement été consacrée à l’étude des différentes approches permettant de corriger automatiquement l’inclinaison d’une image avant les étapes de segmentation et de reconnaissance. Baptiste a comparé plusieurs méthodes, notamment les méthodes basées sur les gradients, la *transformée de Hough*, ainsi que les projections horizontales. Cette analyse a mis en évidence que les approches contenant l’analyse des contours et les projections étaient les plus adaptées dans notre contexte car elles offrent un bon compromis entre précision, robustesse et compatibilité avec la bibliothèque `MagickWand`, utilisée dans l’ensemble des fonctions pour charger, transformer et exporter efficacement les pixels. Ces conclusions ont servi de base à la planification de deux algorithmes complémentaires qui seront développés par la suite.

Interface Utilisateur (Thibault)

Cette semaine, Thibault a pris en charge l’intégration complète de l’interface utilisateur de l’application, un travail central pour le bon fonctionnement global du système. Il a notamment conçu et mis en place des fonctions de rappel (`callback functions`), permettant d’assurer une communication efficace et cohérente entre les différentes composantes du programme, qu’il s’agisse de la logique métier, des modules de traitement ou des éléments graphiques. Grâce à cette architecture basée sur des mécanismes événementiels, les actions de l’utilisateur peuvent être correctement capturées, traitées et répercutées rapidement dans le programme. Ce travail a ainsi contribué à renforcer les échanges entre les modules et à garantir une interaction intuitive, stable et simple pour l’utilisateur final, tout en améliorant la maintenabilité et l’évolutivité de l’interface utilisateur.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a réalisé la mise en place complète de la structure `ConvLayer`, incluant la définition des paramètres configurables (nombre de filtres (`n_filters`), dimensions de noyaux (`height`, `width`), `stride`, `padding`) ainsi que le calcul automatique des dimensions de sortie. Il a également procédé à l'allocation mémoire des différents composants tels que les `kernels`, `biais`, sorties et cache d'entrée, avec une initialisation de type `He` adaptée à l'activation `ReLU`. Le *forward convolutionnel* a été implémenté sous la forme d'une convolution directe, sans `im2col`, à l'aide de boucles imbriquées parcourant successivement les filtres, les positions dans l'espace, les canaux et les éléments du noyau, avec application de `ReLU` en ligne. Des tests préliminaires sur `MNIST` ont confirmé le bon fonctionnement du *forward*, validant ainsi cette première étape du module convolutionnel.

À FAIRE :

- Algorithme d'Upscale (Gaël) : Finaliser l'algorithme.
- Rotation Automatique (Baptiste) : Commencer l'implémentation de l'algorithme avec des gradients.
- Réseau de Neurones (Ethan) : Implémenter la backpropagation.

2.8 Semaine du 10 novembre 2025

Algorithme d'Upscale (Gaël)

Cette semaine, Gaël a poursuivi le travail sur l'algorithme d'**upscale** en **Python**, en améliorant la fiabilité du programme avant sa traduction en langage **C** et en ajoutant plusieurs fonctionnalités nécessaires à son intégration dans le projet. Parmi ces fonctionnalités, on peut citer la gestion de quelques cas d'erreur comme la gestion explicite des cas limites, quand la matrice est plus grande que la taille cible ou la vérification de la validité des indices générés par l'algorithme. De plus, Gaël a ajouté une suite de tests permettant de valider l'algorithme sur différents exemples de lettres stylisées en partie écrites à la main et à l'aide d'outils de génération de matrices aléatoires en ligne. À la fin de la semaine, la version **C** de l'algorithme est considérée comme stable, on peut donc commencer sereinement la traduction en langage **C** la semaine prochaine.

Rotation Automatique (Baptiste)

Cette semaine, Baptiste a implémenté la fonction `detect_rotation_angle_from_edges()`. Cette première méthode repose sur l'intuition que, lorsqu'une image est correctement orientée, les lignes du contour génèrent un gradient particulier maximisant la variance des intensités. L'algorithme clone l'image, lui applique un filtre de contours via `MagickEdgeImage()`, puis effectue une rotation d'essai entre -45° et $+45^\circ$ avec un pas de 1° . Pour chaque rotation, la variance des intensités est évaluée grâce à une analyse pixel par pixel du canal rouge. L'angle maximisant cette variance est ensuite retenu comme orientation la plus probable. Bien que fonctionnelle et assez précise, cette technique s'est révélée coûteuse avec une complexité proportionnelle à la largeur (*Width*) et hauteur (*Height*) de

$$91 \cdot O(W \times H) = O(W \times H)$$

Cette complexité, bien que seulement proportionnelle aux dimensions de l'image, est en réalité importante à cause du facteur multiplicatif énorme, dû au nombre d'angles testés, au coût de chaque rotation et du traitement intensif sur les pixels. Ces limites ont motivé la conception d'une méthode plus rapide.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a implémenté l'ensemble de la *backpropagation* pour la couche de convolution, permettant de compléter le fonctionnement du module convolutif. Il a d'abord ajouté le calcul du gradient des biais, basé sur la somme des gradients de sortie par filtre avec prise en compte de la dérivée de **ReLU**, puis le calcul du gradient des *kernels* en accumulant, pour chaque position, le produit entre le gradient de sortie et les valeurs de `input_cache`. Le gradient d'entrée a également été implémenté de manière optionnelle, via la convolution du gradient de sortie avec les *kernels* retournés, tout en gérant correctement les effets du `padding` et du `stride`. L'ensemble a ensuite été intégré dans la structure `NeuralNetwork`, avec gestion d'un tableau de couches convolutionnelles, du chaînage vers les couches denses via un *flatten buffer*, et la création d'un pipeline complet `conv+flatten+denses`. Les tests réalisés ont montré un premier résultat fonctionnel : un **CNN** composé d'une couche convolutionnelle suivie de deux couches denses atteint environ 75% de précision sur **MNIST** dès la première époque.

À FAIRE :

- Algorithme d'Upscale (Gaël) : Finaliser l'algorithme en langage **C**.
- Rotation Automatique (Baptiste) : Optimiser l'algorithme avec projection.
- Réseau de Neurones (Ethan) : Mettre en place la fonction de coût.
- Détection des Caractères (Thibault) : Continuer l'implémentation de l'algorithme.

2.9 Semaine du 17 novembre 2025

Algorithme d'Upscale (Gaël)

Cette semaine a été consacrée au développement de la fonction d'agrandissement d'image (**upscale**) permettant d'augmenter la résolution d'une matrice binaire représentant un caractère, cette fois-ci en langage C, la version en Python étant fonctionnelle. Pour rappel, l'objectif était d'obtenir une matrice de taille fixe (Dans notre cas 24×24) pour normaliser les entrées destinées aux étapes suivantes du pipeline (segmentation, reconnaissance de caractères, réseau de neurones). Le code implémente une version simple de l'**upscale** par échantillonnage *nearest-neighbor*, où chaque pixel de la matrice finale copie la valeur du pixel correspondant dans la matrice d'origine, selon un ratio d'échelle horizontal et vertical. Une vérification sur la taille de la matrice d'entrée a été ajoutée dans le fichier C de la même manière que dans le fichier Python pour éviter les dépassements.

Rotation Automatique (Baptiste)

Cette semaine, Baptiste a poursuivi son travail sur la rotation automatique et a contribué au développement de la méthode optimisée `detect_rotation_angle_projection()`, destinée à accélérer drastiquement le traitement tout en améliorant la précision. L'algorithme commence par réduire la taille de l'image⁹ afin de limiter le coût de calcul, puis convertit l'ensemble en niveaux de gris pour faciliter l'exportation des données via `MagickExportImagePixels()`. Une recherche de type *coarse-to-fine* a été mise en place : C'est à dire un balayage initial de $\pm 30^\circ$ avec un pas de 2° et une extension éventuelle à $\pm 45^\circ$ si le score est faible, suivi d'un affinement final autour du meilleur angle avec un pas très faible de $0,25^\circ$. Pour chaque rotation testée, l'image est projetée ligne par ligne et la variation absolue entre intensités successives est mesurée. Une variation plus forte correspond à une meilleure orientation. Les premiers tests montrent une nette amélioration des performances : rapidité accrue, précision supérieure, et charge de calcul considérablement réduite.

Figure 2.11: Comparaison des deux algorithmes.

Algorithme	Complexité	Précision
Edges	$O(W \times H)$	1°
Projection	$O(R^2)$	0.25°

avec⁹

$$\begin{cases} W \text{ et } H : \text{Dimensions de l'image originale.} \\ W' \text{ et } H' : \text{Dimensions de l'image réduite.} \\ R = \max(W', H') \leq 400 \end{cases}$$

En conclusion, la méthode basée sur l'analyse des contours présente une complexité proportionnelle à la taille complète de l'image, ce qui la rend coûteuse lorsque l'on explore finement l'intervalle d'angles. À l'inverse, la méthode par projection opère sur une image réduite et limite ainsi son coût à $O(R^2)$, bien plus faible que $O(W \times H)$. Malgré une précision angulaire plus fine, elle reste nettement plus efficace en pratique. Cette différence de complexité justifie pleinement l'adoption de la méthode optimisée pour la rotation automatique.

⁹L'image réduite a une taille maximale de 400px

Algorithme de Résolution (Thibault)

Ces deux dernières semaines, Thibault s'est consacré à l'assemblage et à la structuration complète de l'algorithme de résolution, une étape clé du projet consistant à lier tous les modules du projet en une suite logique d'étapes. Cette chaîne débute par la capture de l'image, qui constitue l'entrée principale de l'algorithme, puis se poursuit par l'application successive des différents traitements d'image afin d'améliorer la qualité des données et d'en extraire les informations pertinentes. Ces étapes sont les algorithmes implémentés par Baptiste : **Grayscale**, **Rotation d'Image** et **Suppression du Bruit**. Une fois ces traitements effectués, on extrait les données grâce aux modules dédiés et on procède ensuite à la résolution proprement dite, en exploitant les données extraites pour produire un résultat exploitable, en utilisant l'algorithme **Solver** implémenté par Gaël. Enfin, Thibault a veillé à l'affichage clair et compréhensible du résultat final à l'utilisateur, avec l'interface utilisateur implémentée précédemment.

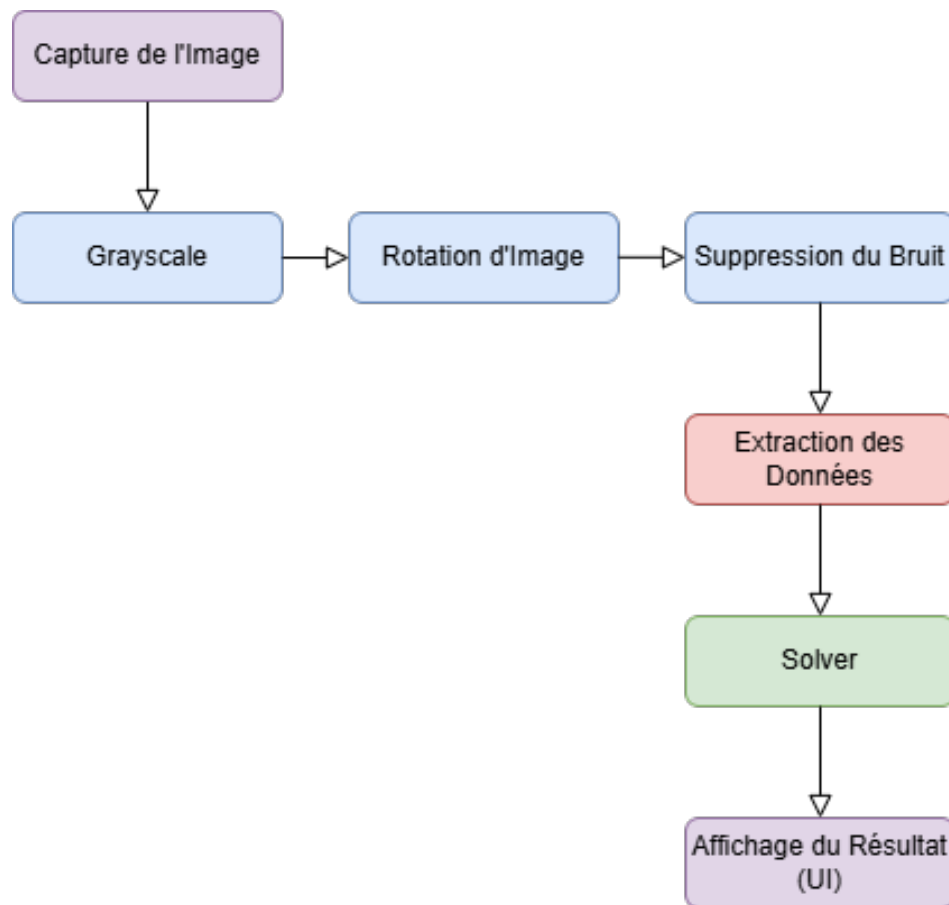


Figure 2.12: Schéma de la résolution (le prétraitement en bleu, l'interface utilisateur en violet).

Réseau de Neurones (Ethan)

Cette semaine, Ethan a introduit un nouveau format de persistance `.nn` destiné à stocker des architectures complètes comprenant plusieurs couches convolutionnelles. Ce format inclut une en-tête **MAGIC**, le nombre de couches convolutionnelles, la configuration détaillée de chacune d'entre elles (dimensions d'entrée, nombre de filtres, tailles de noyaux, *stride*, *padding*), ainsi que l'ensemble des *kernels*, *biais* et paramètres des couches denses. Les fonctions `save_conv_layer()` et `load_conv_layer()` ont été correctement implémentées, assurant une sérialisation fiable. De plus, Ethan a ajouté la prise en charge du multi-convolution, avec la création d'un réseau comportant deux couches convolutionnelles successives (de 8 puis 16 filtres), tout en supprimant le *pooling* du format et du code afin de simplifier la structure. Les premières expérimentations montrent qu'un réseau à deux convolutions atteint environ 85% de précision sur MNIST dès la première époque, confirmant le bon fonctionnement du format et de la nouvelle architecture.

2.10 Semaine du 24 novembre 2025

Algorithme d’Upscale (Gaël)

Cette semaine, Gaël a poursuivi l’implémentation de la fonction `upscale`, en testant différentes tailles et configurations de matrices pour garantir la fiabilité de l’algorithme. Plusieurs tests ont été réalisés, notamment sur de petites matrices 3×3 générées dynamiquement. Les affichages console ont permis de confirmer que les proportions étaient correctement préservées et que les indices ne sortaient pas des bornes. Gaël a également documenté le fonctionnement du fichier, ainsi que les contraintes d’utilisation (dimensions max, comportement aux limites) dans le code. Une attention particulière a été portée à la gestion mémoire et à la compatibilité avec les autres modules déjà écrits par le groupe, notamment la détection des zones et la segmentation. Un premier travail d’intégration a été amorcé, mais l’interconnexion complète sera réalisée une fois que les autres modules (cleaning, rotation, segmentation) auront été terminés.

```
gael@OrdinateurGaelKerninon:~/OCR-KeltIA/KeltIA/deno$ ./upscale | tr '0' '.'
===== INPUT ===== (3x3)
. 1 .
1 . 1
. 1 .

===== OUTPUT ===== (24x24)
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 . . . . . 1 1 1 1 1 1 1 1
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
. . . . . 1 1 1 1 1 1 1 1 . . . . .
```

Figure 2.13: Exemple d’aggrandissement d’une matrice 3×3 en 24×24 . ($\lambda = 8$)

Réseau de Neurones (Ethan)

Cette semaine, Ethan s’est consacré à l’intégration de `datasets` et à la mise en place d’un entraînement de niveau industriel. Il a implémenté un `loader` binaire compatible avec le format `IDX`, incluant les fonctions `load_mnist_images()`¹⁰ ainsi que `emnist_letters_to_dataset`, permettant de convertir les labels EMNIST Letters de l’intervalle $[1 - 26]$ vers $[0 - 25]$ afin d’obtenir un ensemble de 26 classes correspondant aux lettres majuscules $[A - Z]$. Les images sont systématiquement normalisées¹¹ avant utilisation. Ethan a également implémenté un entraînement `SGD` robuste avec `batch_size=1`, dans lequel la rétropropagation est immédiatement suivie de la mise à jour des paramètres et de la réinitialisation des gradients. Les fonctions `update_conv_parameters()` ont été stabilisées, un gestionnaire `SIGINT` a été ajouté pour permettre une interruption propre, et une progression en direct est affichée tous les 1000 échantillons, accompagnée de journaux de performance. Les premiers résultats sont concluants : *EMNIST Letters* est désormais pleinement fonctionnel, *MNIST* atteint environ 92% de précision à la deuxième époque, et l’entraînement peut être interrompu proprement à tout moment.

¹⁰Lecture `big-endian`, vérification des valeurs `MAGIC 0x00000803/01`

¹¹Cf. Algorithme d’Upscale

Rotation Automatique (Baptiste)

Cette semaine, Baptiste s'est consacré à l'intégration du module complet de rotation automatique d'image dans la chaîne de pré-traitement. La fonction `auto_rotate_image()` a été finalisée : elle exploite l'angle détecté par la méthode de projection, puis applique la correction définitive via `rotate_image()`. Des essais ont été menés par Baptiste sur un ensemble varié d'images : grilles inclinées, contenus bruités, luminosité variable. Les résultats sont très satisfaisants : l'orientation est corrigée de manière fiable, et l'ensemble du module est désormais stable et prêt pour une utilisation dans le système global.

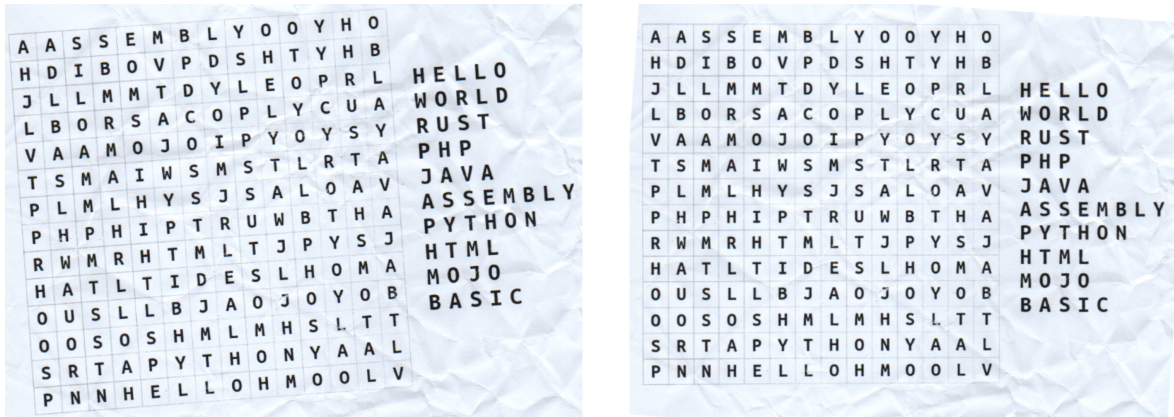


Figure 2.14: Exemple d'Application de l'Algorithme de Rotation Automatique (Avant / Après)

À FAIRE :

- Rotation Automatique (Baptiste) : Développer une version **Brute Force**.
- Réseau de Neurones (Ethan) : Refactoriser le code pour éliminer CBLAS.
- Interface Utilisateur (Thibault) : Intégrer les options liées à l'entraînement du modèle.
- Documents de Projet (Gaël) : Rédiger le rapport.

2.11 Semaine du 1 décembre 2025

Rotation Automatique (Baptiste)

Pour compléter le système de rotation automatique d'image, une version brute-force paramétrable `auto_rotate_image_bruteforce()`, a été développée. Elle permet d'effectuer une recherche exhaustive selon un pas défini, utile dans des cas extrêmes (forte inclinaison ou bruit important). Un programme de démonstration indépendant a également été produit : il charge une image, calcule son angle optimal, puis génère automatiquement la version redressée. Ce programme constitue un support important pour illustrer la capacité du programme lors de la soutenance et facilitera les tests à l'avenir.

Interface Utilisateur (Thibault)

Pendant cette semaine, Thibault a réalisé des derniers ajouts sur l'interface utilisateur spécifiquement dédiés à la phase d'entraînement du modèle. Cette partie de l'interface a été pensée afin de simplifier la configuration et le lancement des sessions d'apprentissage, notamment en permettant à l'utilisateur de sélectionner facilement un répertoire de données en entrée. Elle offre également un affichage en temps réel du déroulement du processus d'entraînement, rendant visibles les différentes étapes, l'avancement global ainsi que les éventuelles informations intermédiaires pertinentes. Cette visualisation facilite le suivi, l'analyse et la validation des phases d'apprentissage, tout en améliorant l'ergonomie de l'outil et l'expérience utilisateur lors de l'entraînement du modèle.

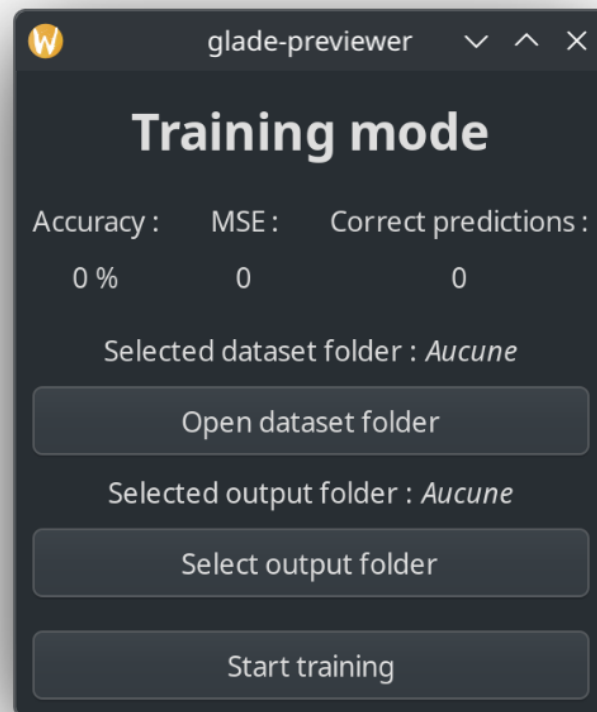


Figure 2.15: Interface Utilisateur pour l'Entraînement.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a effectué un important travail de refactorisation visant à éliminer toute dépendance à CBLAS, afin de garantir une portabilité totale du code. Les opérations matricielles ont été entièrement réécrites manuellement, incluant le calcul de `delta_hidden_layer` via des boucles explicites pour l'opération :

$$W^T \delta$$

Ainsi que pour le calcul des gradients de poids par produit extérieur et la somme des gradients de biais. Le chaînage convolution $\rightarrow$ dense a également été entièrement réimplémenté sans assistance BLAS, en procédant à un calcul direct de $W^T \delta$. Parallèlement, Ethan a mis en place une série de tests exhaustifs afin d'assurer la robustesse du réseau : `test_cnn_save_load`¹², `test_stride_padding`¹³, `test_multi_conv`¹⁴, ainsi qu'un test de comparaison `bit-perfect` garantissant la cohérence totale après sauvegarde et rechargement. Un important travail de nettoyage a également été effectué, incluant la suppression complète du *pooling* dans l'ensemble des fichiers et la simplification du `Makefile` pour fonctionner sans `OpenBLAS`. Les performances finales sont très positives : le réseau atteint 83% de précision dès la première époque sur *EMNIST Letters* et progresse jusqu'à 91% après cent époques. Sur l'ensemble des cinq semaines, le modèle est passé de 20% à 91% de précision sur un problème à 26 classes, aboutissant à un CNN portable, fiable, exhaustivement testé et prêt pour un usage en production.

À FAIRE :

- Documents de Projet (Gaël) : Rédiger le rapport.
- Rotation Automatique (Baptiste) : Débogage.
- Interface Utilisateur (Thibault) : Débogage.
- Réseau de Neurones (Ethan) : Débogage.

¹²Tests réalisés avec des paramètres aléatoires.

¹³Avec *stride* $\in \{1, 2\}$ et *padding* $\in \{1, 2, 3\}$

¹⁴Deux couches convolutionnelles distinctes

2.12 Semaine du 8 décembre 2025

Interface Utilisateur (Thibault)

Lors de cette dernière semaine, Thibault s'est principalement consacré aux phases de débogage et de stabilisation du programme, en s'appuyant sur l'ensemble des fonctionnalités développées au niveau de l'interface utilisateur. L'interface utilisateur, qui centralise la sélection des données d'entrée, le pilotage des traitements, l'affichage des résultats et le suivi en temps réel des processus, a ainsi été testée de manière approfondie afin de garantir sa cohérence. Cette phase de débogage a permis d'assurer une communication logique entre les différents modules, d'améliorer la fluidité de l'interface, et de valider le bon déroulement de l'ensemble des fonctionnalités offertes à l'utilisateur, contribuant ainsi à la fiabilité du programme.

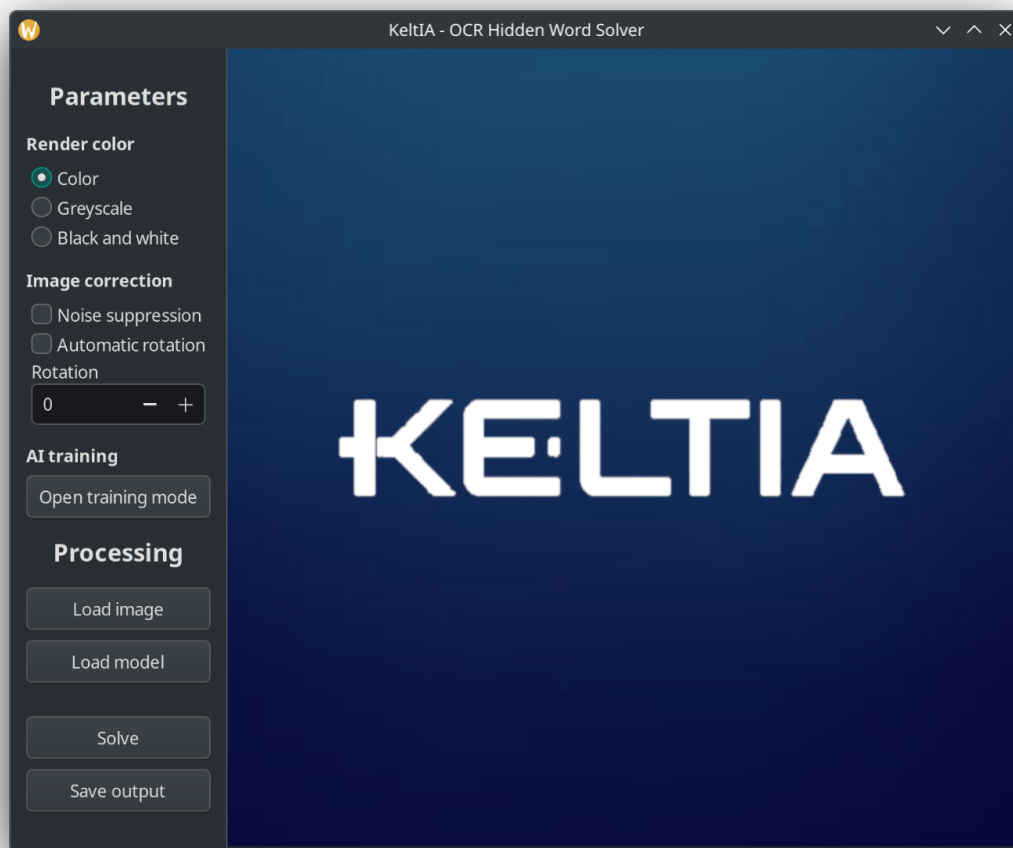


Figure 2.16: Capture d'écran de l'Interface Utilisateur.

Documents de Projet (Gaël)

Cette semaine a été consacrée à la rédaction des documents de projet, et en particulier du rapport de projet en $\text{\LaTeX}$. Le rapport a été complété avec les remarques soulevées par le jury lors de la dernière présentation, comme l'ajout d'une frise chronologique reprenant les temps théoriques et factuels de l'implémentation du projet dans les temps, et évidemment les rapports individuels hebdomadaires rédigés à la main à partir des notes de chaque membre du groupe notées régulièrement sur un forum privé en commun. Cette auto discipline de chaque membre du groupe a permis de faciliter la rédaction du rapport à la main et donc d'en garantir son authenticité et sa formalité tant dans la forme que dans le fond.

Rotation Automatique (Baptiste)

Cette semaine, Baptiste s'est concentré sur une phase de débogage du module de rotation automatique d'image. Il a identifié plusieurs comportements ambigus liés à la gestion des angles les plus extrêmes, à l'exportation des pixels et à la cohérence des résultats entre les deux méthodes de détection. Ces corrections ont permis d'améliorer la stabilité du module, d'assurer une meilleure polyvalence face à différents types d'images et de préparer une intégration plus fiable dans la chaîne algorithmique de pré-traitement.

Réseau de Neurones (Ethan)

Cette semaine, Ethan a contribué au débogage approfondi de la dernière version du système. Plusieurs vérifications ont été menées pour garantir la cohérence entre les différentes configurations de convolution, le chargement et la sauvegarde des modèles et l'entraînement sur jeux de données réels. Les corrections ont porté aussi bien sur des erreurs d'indexation que sur des incohérences de dimensions ou de réinitialisation de gradients. Grâce à ce travail de stabilisation, la version actuelle est désormais fiable, reproductible et prête pour les tests finaux de performance. De plus, on peut également mentionner une évolution notable de la précision (*Accuracy*) du modèle :

Semaine	Dataset	Accuracy	Epoch
03/11	MNIST	75%	1
10/11	MNIST	85%	1
24/11	MNIST	92%	2
01/12	EMNIST Letters	83%	1
01/12	EMNIST Letters	91%	100

Table 2.1: Évolution des performances du réseau de neurones au fil du temps

Part III

Bibliographie et Sources

Chapter 3

Bibliographie et Sources

3.1 Structures de Données

- Stacks
- Réseaux de Neurones
- Couches Convolutionnelles
- Format de persistance : `.nn`.

3.2 Algorithmes

- Algorithme de N. Otsu¹, 1979.
- Tutoriel vidéo : *Backpropagation Algorithm Explained*, 3Blue1Brown (YouTube), 2021.
- Tutoriel : *Word Search Solver Implementation in C*, GeeksForGeeks, 2023.
- Algorithme d'Upscale (*Nearest-Neighbor*): Utilisé pour la normalisation de la taille des caractères extraits.
- Rétropropagation du Gradient (*Back-Propagation*): Algorithme clé pour l'entraînement du Réseau de Neurones (MLP et CNN).
- Projections Horizontales (Rotation Automatique)
- Rotation par force brute (Rotation Automatique)
- Transformée de Hough (1959) et Analyse des contours.
- Algorithme de Remplissage par Diffusion (*Flood Fill*).
- Implémentation Manuelle des Opérations Matricielles.

¹A Threshold Selection Method from Gray-Level Histograms

3.3 Tutoriels et Ressources Vidéo

- Vidéo : *Otsu Thresholding Algorithm Tutorial*, DigitalSreeni (YouTube), 2022.
- Vidéo : *Makefile Basics for C Projects*, Jacob Sorber (YouTube), 2023.
- Documentation : *Managing Git Hooks for CI/CD*, Atlassian Developer Docs, consultée en 2025.
- Vidéo : *Implementing Stacks and DFS in C*, CodeVault (YouTube), 2023.

3.4 Librairies

- Documentation : *MagickWand API for C*, version 7.1.1, consultée en 2025.
- Documentation : *Glade User Interface Designer*, version 3.38, consultée en 2025.
- Documentation : *GTK+ 3.0 Reference Manual*, version 3.24, consultée en 2025.
- Documentation : *GSL CBLAS Library*, version 2.8, consultée en 2025.

Conclusion

Conclusion

Le projet **OCR-KeltIA** constitue une expérience riche et formatrice, tant sur le plan technique que méthodologique. Il nous permet d'aborder des thématiques complexes telles que la **reconnaissance optique de caractères**, le **traitement d'images** et l'implémentation d'un **réseau de neurones artificiels** en langage C. L'objectif principal étant concevoir une application capable d'analyser et de résoudre automatiquement une grille de mots cachés à partir d'une image, ce projet a d'abord nécessité une approche rigoureuse, une conception réfléchie et une coordination entre les membres du groupe.

Le développement en langage C nous a conduits à porter une attention particulière à la gestion de la mémoire, aux performances et à la structure du code, ce qui nous permet d'améliorer nos compétences dans ce langage. D'autre part, la mise en place de **Makefile**, de scripts d'automatisation et de *Git hooks* a permis d'assurer la cohérence du dépôt et la standardisation du formatage, facilitant ainsi le travail collaboratif. L'usage d'outils tels que Vim, VSCoDe et le compilateur GCC s'est révélé essentiel pour maintenir un environnement de développement stable et efficace.

Sur le plan algorithmique, la conception d'un réseau de neurones fonctionnel, capable d'apprendre à reconnaître des caractères à partir d'images, a constitué un défi majeur. Nous avons dû expérimenter différentes architectures et fonctions d'activation, tout en assurant la compatibilité avec la structure modulaire du projet. Le système final, combinant la reconnaissance, illustre la proximité entre les domaines du traitement de données visuelles et de l'intelligence artificielle.

En dehors des aspects techniques, ce projet nous a permis de développer des compétences en **gestion de projet**, en **travail d'équipe** et en **résolution de problèmes**. La répartition des tâches, la communication continue avec des réunions régulières et l'utilisation des branches Git pour l'intégration progressive des fonctionnalités ont été des facteurs clés de réussite. Chaque membre a pu apporter sa contribution spécifique tout en participant à une vision commune, celle de produire un logiciel complet et cohérent avec les consignes imposées.

En conclusion, **OCR KeltIA** représente non seulement l'aboutissement d'un travail collectif rigoureux, mais aussi une application concrète des enseignements théoriques reçus. Il illustre notre capacité à concevoir un système complet, du bas niveau jusqu'à l'intelligence artificielle, tout en respectant les contraintes académiques et techniques imposées.